

*"The container packages the application;  
OpenShift manages it at scale."*

## MODULE 8

# Containerization with Red Hat OpenShift

Packaging, deploying, scaling and operating Spring Boot microservices.

Containers vs.  
virtual  
machines

Building  
container  
images

Container  
registries

OpenShift  
architecture

Deploying  
microservices

Configuration  
and secrets

Health checks

Scaling and  
management

# LEARNING OBJECTIVES

*Containers vs. VMs, images, registries, OpenShift concepts, deployment and scaling.*



## BY THE END OF THIS MODULE, YOU CAN

1. Compare containers and virtual machines
2. Explain container images and containers
3. Build a Spring Boot container image
4. Apply container build best practices
5. Explain container registries
6. Describe OpenShift architecture
7. Identify key OpenShift objects
8. Deploy a microservice to OpenShift
9. Externalize configuration and secrets
10. Configure application health checks
11. Scale applications
12. Understand rolling updates and resources



## FOUR SECTIONS

### 1 Container fundamentals

Containers vs VMs

Images

Running containers

### 2 Building & distributing

JAR

Container  
Build

Image

Registry

### 3 OpenShift architecture

Cluster

Control Plane

Worker Nodes

Pods

### 4 Running applications

Deployment

Pods

Service

Route

Scale/Health

**KEY TAKEAWAY** Follow the full path from Spring Boot source to a production workload on OpenShift.

# KEY TERMS FOR THIS MODULE

*Plain definitions for the words we'll use most.*

## 1 Container image

A read-only package of the application, its runtime and its dependencies.

## 2 Container

A running instance of an image, isolated from other processes.

## 3 Multi-stage build

Build in one stage, then copy only the result into a small runtime image.

## 4 Registry

A server that stores and distributes container images.

## 5 Pod

The smallest unit OpenShift runs — one or more containers sharing a network.

## 6 Deployment

Keeps the desired number of pods running and manages rolling updates.

## 7 Service and Route

A Service gives pods a stable internal address; a Route exposes it outside the cluster.

## 8 ConfigMap and Secret

Objects that inject configuration and sensitive values into pods at runtime.

**KEY TAKEAWAY** Agree on the words first — every later slide builds on them.

# FULL FORMS USED IN THIS MODULE

*Every abbreviation on these slides, spelled out.*

|              |                                              |              |                                    |             |                            |
|--------------|----------------------------------------------|--------------|------------------------------------|-------------|----------------------------|
| <b>AI</b>    | Artificial Intelligence                      | <b>HTTP</b>  | HyperText Transfer Protocol        | <b>JWT</b>  | JSON Web Token             |
| <b>API</b>   | Application Programming Interface            | <b>HTTPS</b> | HyperText Transfer Protocol Secure | <b>OOM</b>  | Out Of Memory              |
| <b>CI</b>    | Continuous Integration                       | <b>ID</b>    | Identifier                         | <b>OS</b>   | Operating System           |
| <b>CI/CD</b> | Continuous Integration / Continuous Delivery | <b>IP</b>    | Internet Protocol                  | <b>SBOM</b> | Software Bill of Materials |
| <b>CPU</b>   | Central Processing Unit                      | <b>JAR</b>   | Java ARchive                       | <b>TLS</b>  | Transport Layer Security   |
| <b>CVE</b>   | Common Vulnerabilities and Exposures         | <b>JDBC</b>  | Java Database Connectivity         | <b>UBI</b>  | Universal Base Image       |
| <b>DB</b>    | Database                                     | <b>JDK</b>   | Java Development Kit               | <b>URL</b>  | Uniform Resource Locator   |
| <b>HPA</b>   | Horizontal Pod Autoscaler                    | <b>JRE</b>   | Java Runtime Environment           | <b>YAML</b> | YAML Ain't Markup Language |

**KEY TAKEAWAY** When an abbreviation isn't clear, the full form is on this slide.

# FROM MODULE 7 TO MODULE 8

*The pipeline produces a tested image — now look inside it and the platform that runs it.*

## MODULE 7 GAVE US

- ✓ Git workflows and pull-request gates
- ✓ CI/CD pipelines with tests and scans
- ✓ Build once, promote the same image
- ✓ Blue/Green and canary releases
- ✓ Security in the pipeline

**NOW** Package each service as an image and run it reliably on OpenShift.



## OUR PATH THROUGH MODULE 8

- 1 Containers, VMs and images
- 2 Build a secure Spring Boot image
- 3 Tag, push and scan in a registry
- 4 OpenShift architecture and objects
- 5 Deploy with config, secrets and probes
- 6 Scale, roll out and set resources
- 7 Troubleshoot and see the whole picture

Part 1

Part 2

Part 2

Part 3

Part 4

Part 4

Part 4

**KEY TAKEAWAY** Module 7 automated the delivery; Module 8 is what gets delivered — and where it runs.

# PART 1: CONTAINER FUNDAMENTALS

*What a container really is — and why microservices need one.*

**Part 1**  
Container fundamentals

**Part 2**  
Build & distribute

**Part 3**  
OpenShift architecture

**Part 4**  
Running applications

## SO FAR

- ✓ CI builds and tests every change
- ✓ The pipeline publishes a versioned image
- ✓ Releases promote that same image

## NEXT QUESTION

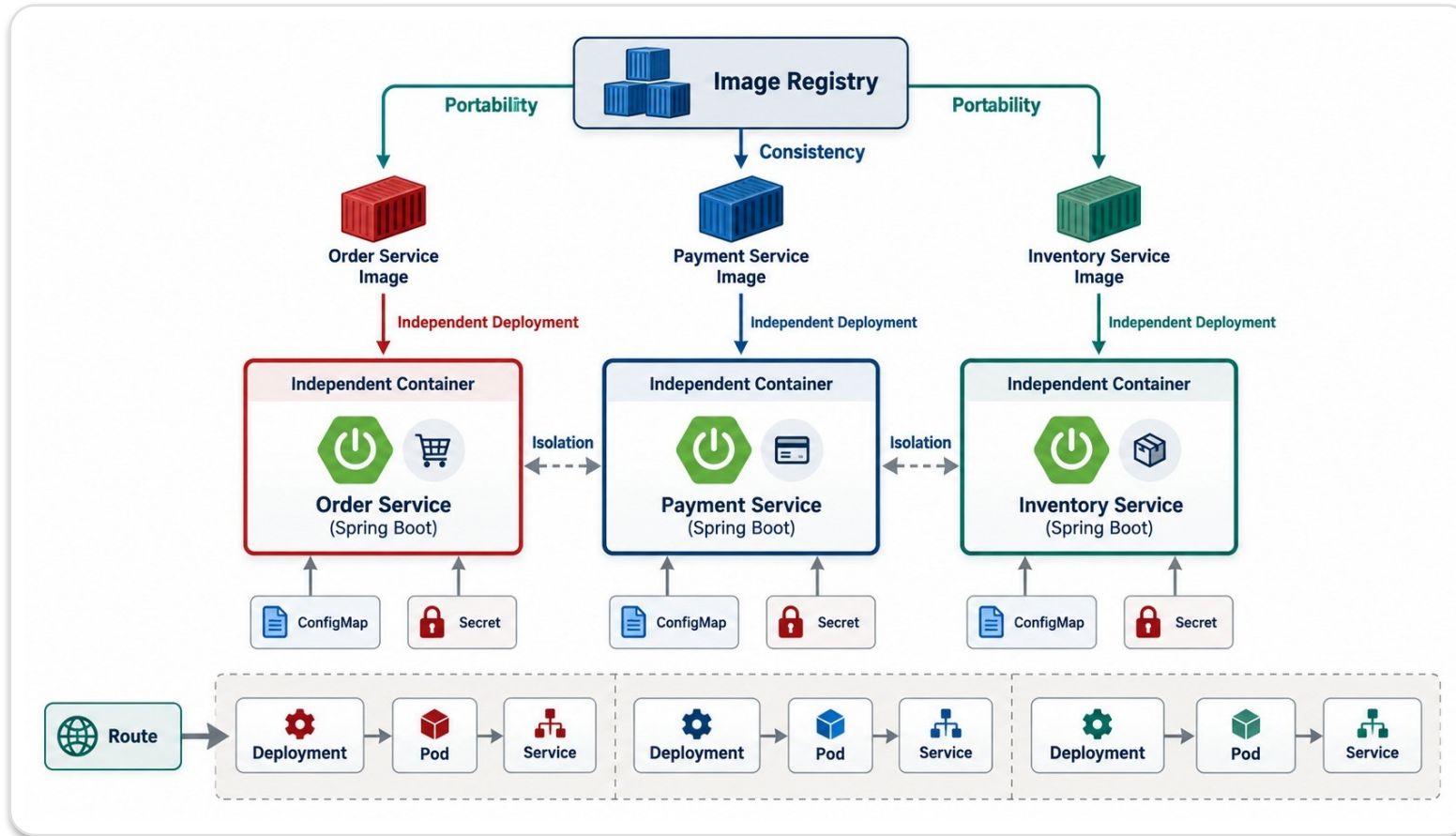
**What exactly is this “image” the pipeline builds — and why use containers at all?**

## IN THIS PART

- 1 Why containers for microservices
- 2 Containers vs. virtual machines
- 3 How containers work
- 4 Images vs. running containers
- 5 Anatomy of an image

# WHY CONTAINERS FOR MICROSERVICES?

*Portability, isolation, consistency, fast startup and independent deployment.*



## “Works on my machine”

Different JDKs, libraries and OS settings make environments drift.

## WHAT A CONTAINER PACKAGES

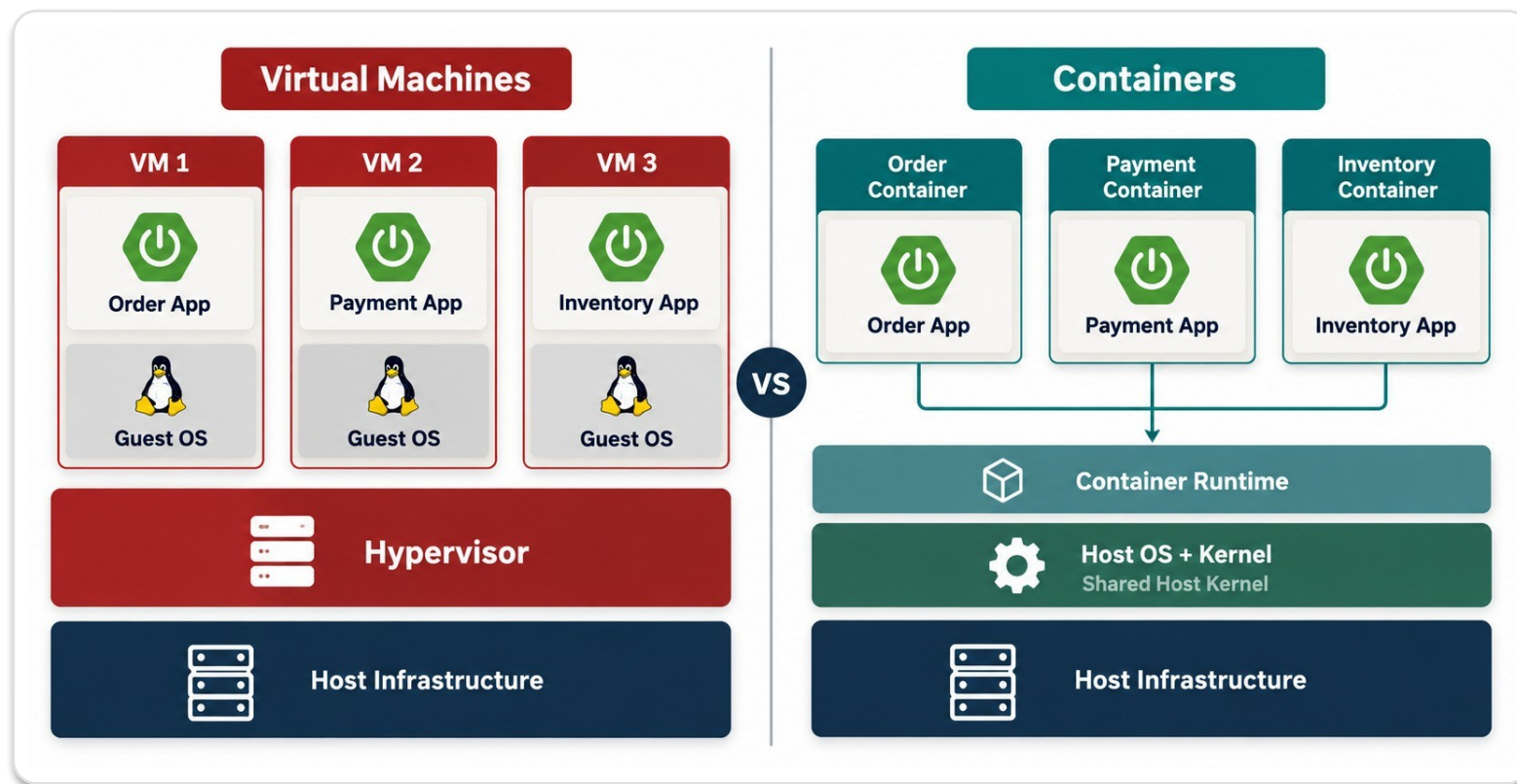
- ✓ The JRE and dependencies
- ✓ The application JAR
- ✓ Startup command and defaults

**FITS MICROSERVICES** Each service versions, deploys and scales alone.

**KEY TAKEAWAY** Containers let each microservice be packaged, versioned, deployed and scaled independently.

# CONTAINERS VS. VIRTUAL MACHINES

*Architecture, startup, resource overhead and isolation.*



## Virtual machine

Full guest OS; boots in minutes; strong isolation; gigabytes.

## Container

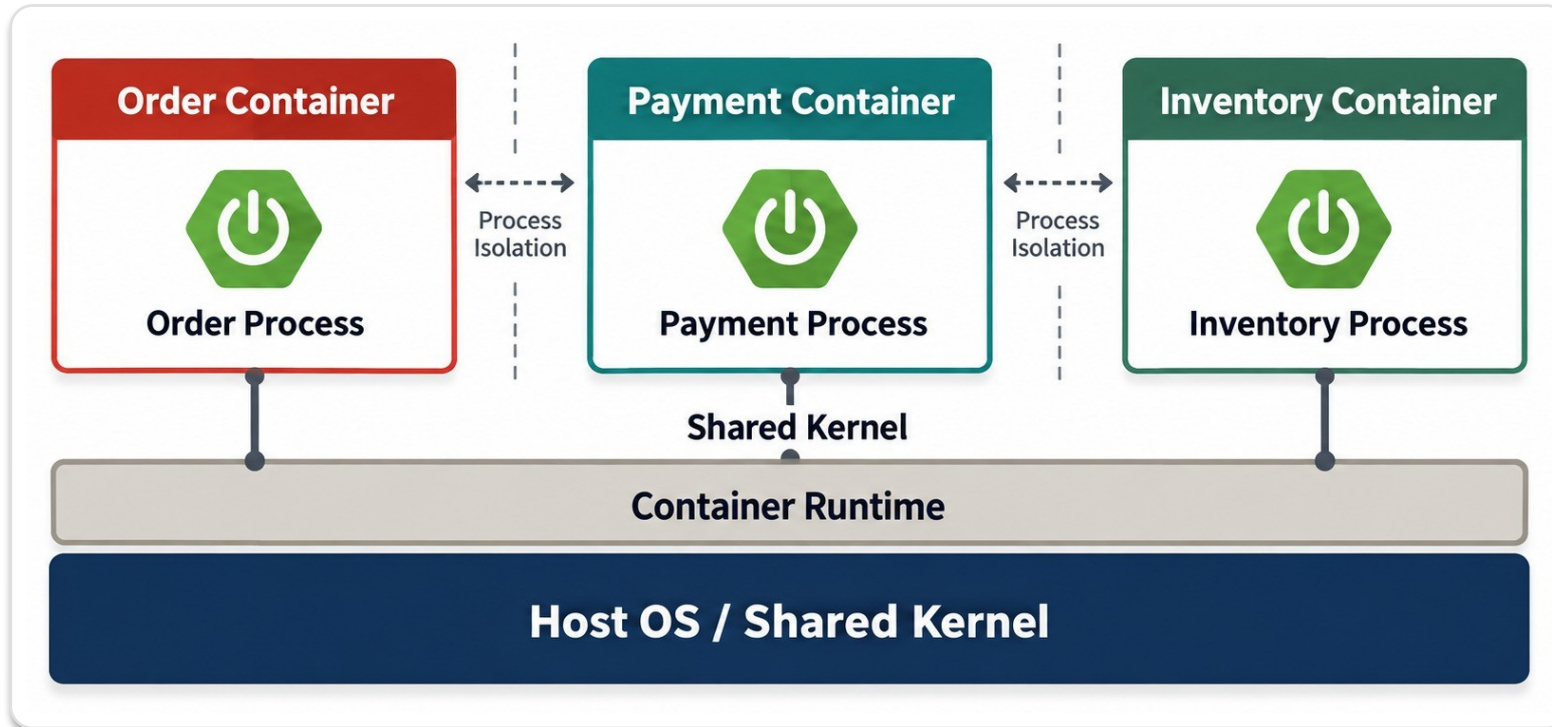
Shares the host kernel; starts in seconds; megabytes.

**NOT A REPLACEMENT** OpenShift worker nodes are often VMs running containers.

**KEY TAKEAWAY** VMs virtualize hardware; containers isolate processes on a shared kernel — and they coexist.

# HOW CONTAINERS WORK

*Application and dependencies sharing the host kernel with process isolation.*



## WHAT MAKES THE ISOLATION

**Namespaces**

**cgroups**

**Layered filesystem**

**Own network**

### Namespaces

Give each container its own view of processes, files and network.

### cgroups

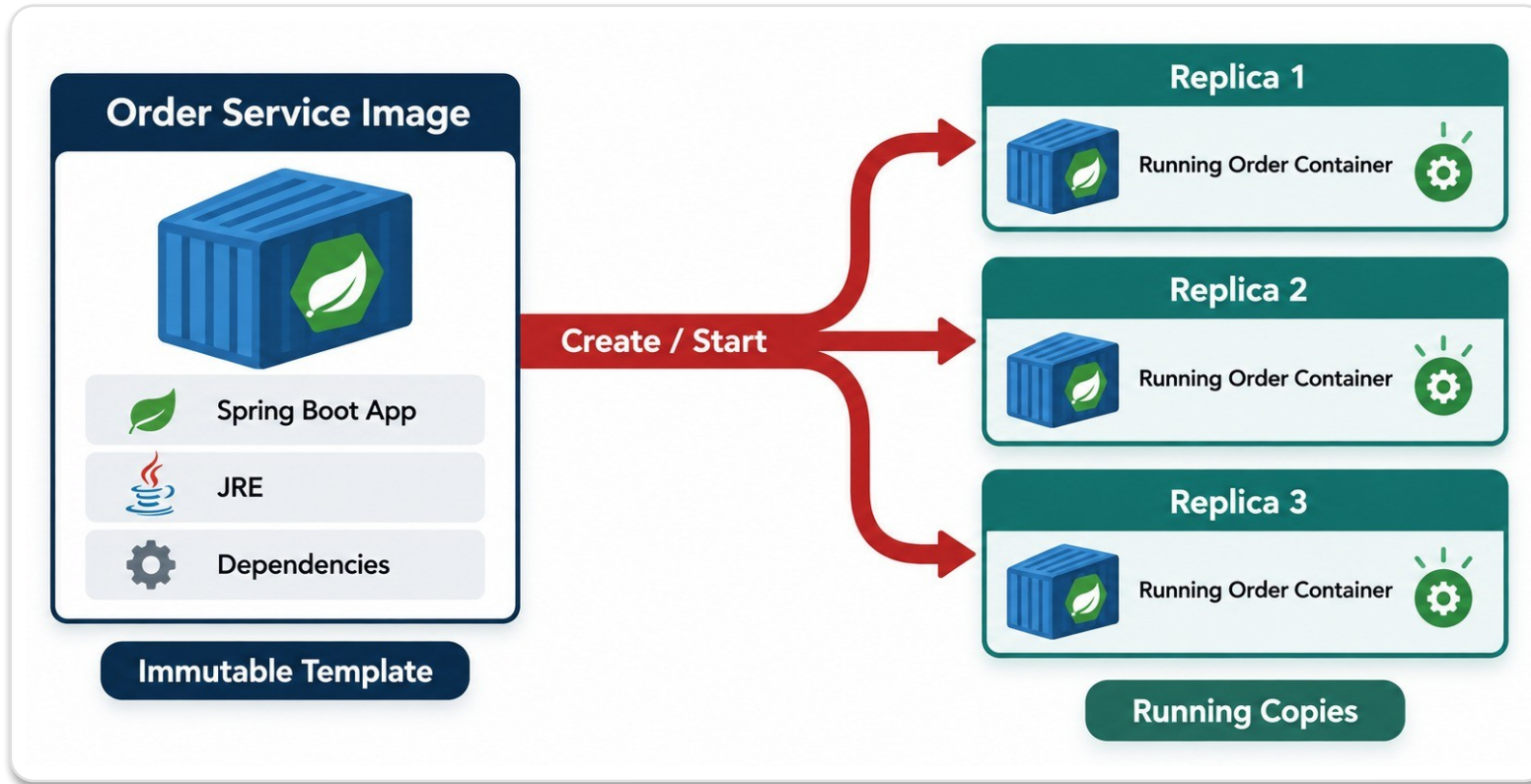
Limit how much CPU and memory each container may use.

**ENOUGH DEPTH** You don't manage these directly on OpenShift.

**KEY TAKEAWAY** A container is an isolated process — with its own view of files, network and resources.

# CONTAINER IMAGE VS. RUNNING CONTAINER

*An immutable image template versus running instances.*



## Image

Read-only, versioned, stored in a registry.

## Container

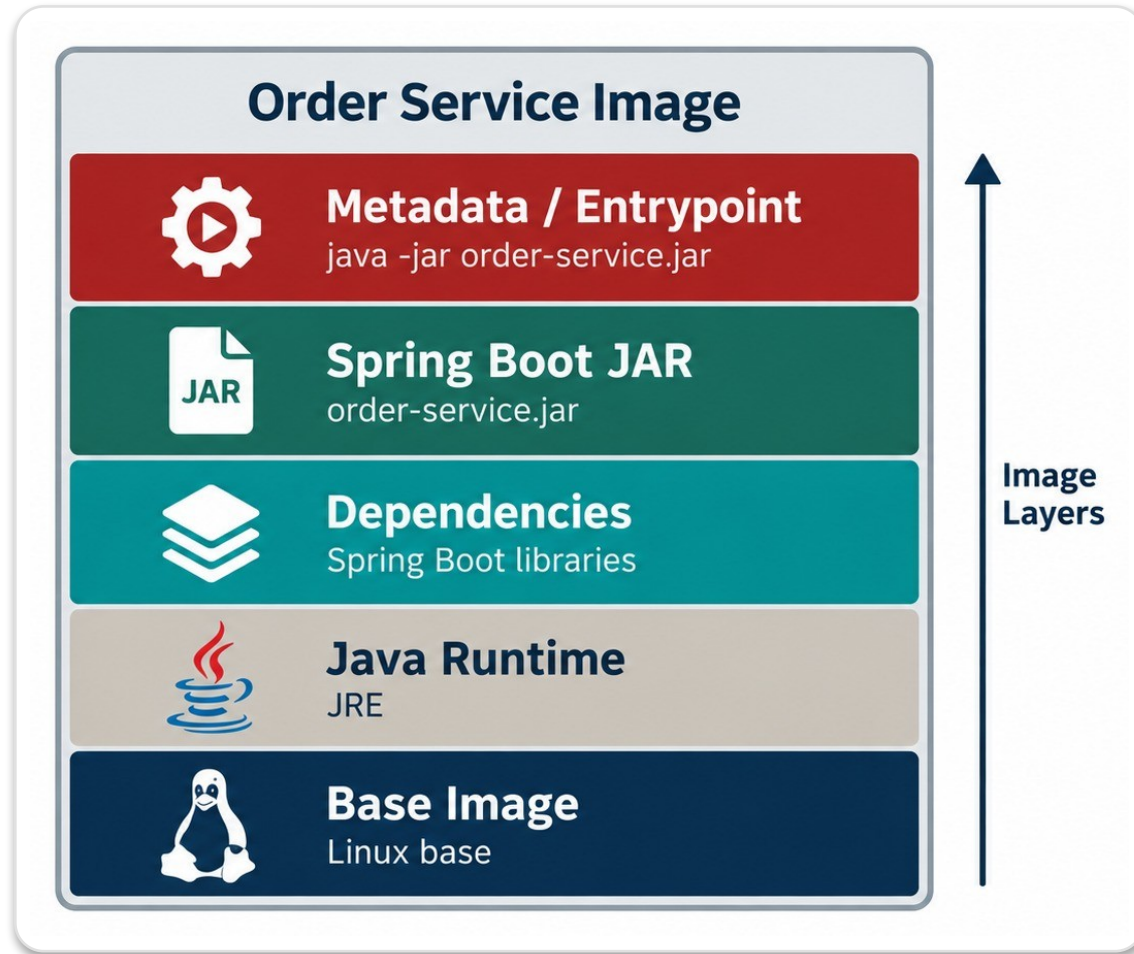
A running instance with its own writable layer — lost when it stops.

**LINK TO SCALING** Three replicas = three containers from one image.

**KEY TAKEAWAY** One image can create many identical running containers.

# ANATOMY OF A CONTAINER IMAGE

*Base image, runtime, application JAR, config defaults, metadata and layers.*



## Why layers matter

Change only your code and only the top layer is rebuilt and pushed.

## Order layers well

Put rarely changing things first: base, JRE, dependencies, then the app.

**KEEP IT SMALL** No build tools, caches or test files in the final image.

**KEY TAKEAWAY** Layered images reuse unchanged layers — so only what changed gets rebuilt and moved.

# PART 2: BUILDING AND DISTRIBUTING IMAGES

*From a Spring Boot JAR to a trusted image in a registry.*



## SO FAR

- ✓ Containers isolate processes on a shared kernel
- ✓ One image runs as many containers
- ✓ Images are built from reusable layers

## NEXT QUESTION

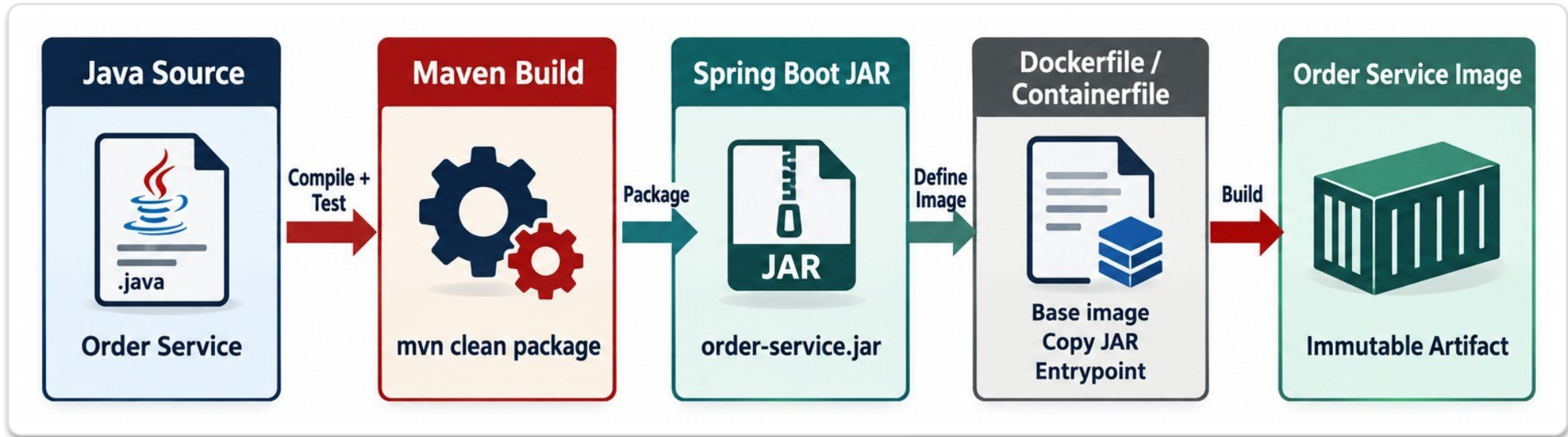
**How do we build a small, secure image — and get it safely to the cluster?**

## IN THIS PART

- 1 The Spring Boot build flow
- 2 A multi-stage, non-root Containerfile — in code
- 3 Good vs. bad images
- 4 Exercise 4.2: containerize Account Service
- 5 Tags, registries and scanning
- 6 Exercise: review a Containerfile

# BUILDING A SPRING BOOT CONTAINER

*JAR → Dockerfile/Containerfile → build → image.*



## Build and run locally

```
./mvnw -B package
podman build -t order-service:1.0.0 .
podman run -p 8080:8080 order-  
service:1.0.0
```

## Docker or Podman

Same Containerfile syntax; Podman is common in Red Hat environments.

**BASE IMAGES** Use trusted, maintained images such as UBI or Temurin.

**KEY TAKEAWAY** Build the JAR, describe the image, then build it into a versioned container image.

# A MULTI-STAGE, NON-ROOT CONTAINERFILE

*Compile in a builder stage; ship only the JRE and the JAR, running as a normal user.*

## ✓ Containerfile

```
# ---- build stage ----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY mvnw pom.xml ./
COPY .mvn .mvn
RUN ./mvnw -B dependency:go-offline          # cached layer
COPY src src
RUN ./mvnw -B package -DskipTests

# ---- runtime stage ----
FROM eclipse-temurin:21-jre
RUN useradd --uid 1001 --gid 0 md287
WORKDIR /app
COPY --from=build --chown=1001:0 /src/target/*.jar app.jar
USER 1001
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

### Two stages

Maven and the JDK stay in the builder — not in production.

### Dependency layer

Unchanged dependencies are reused on every rebuild.

### Non-root

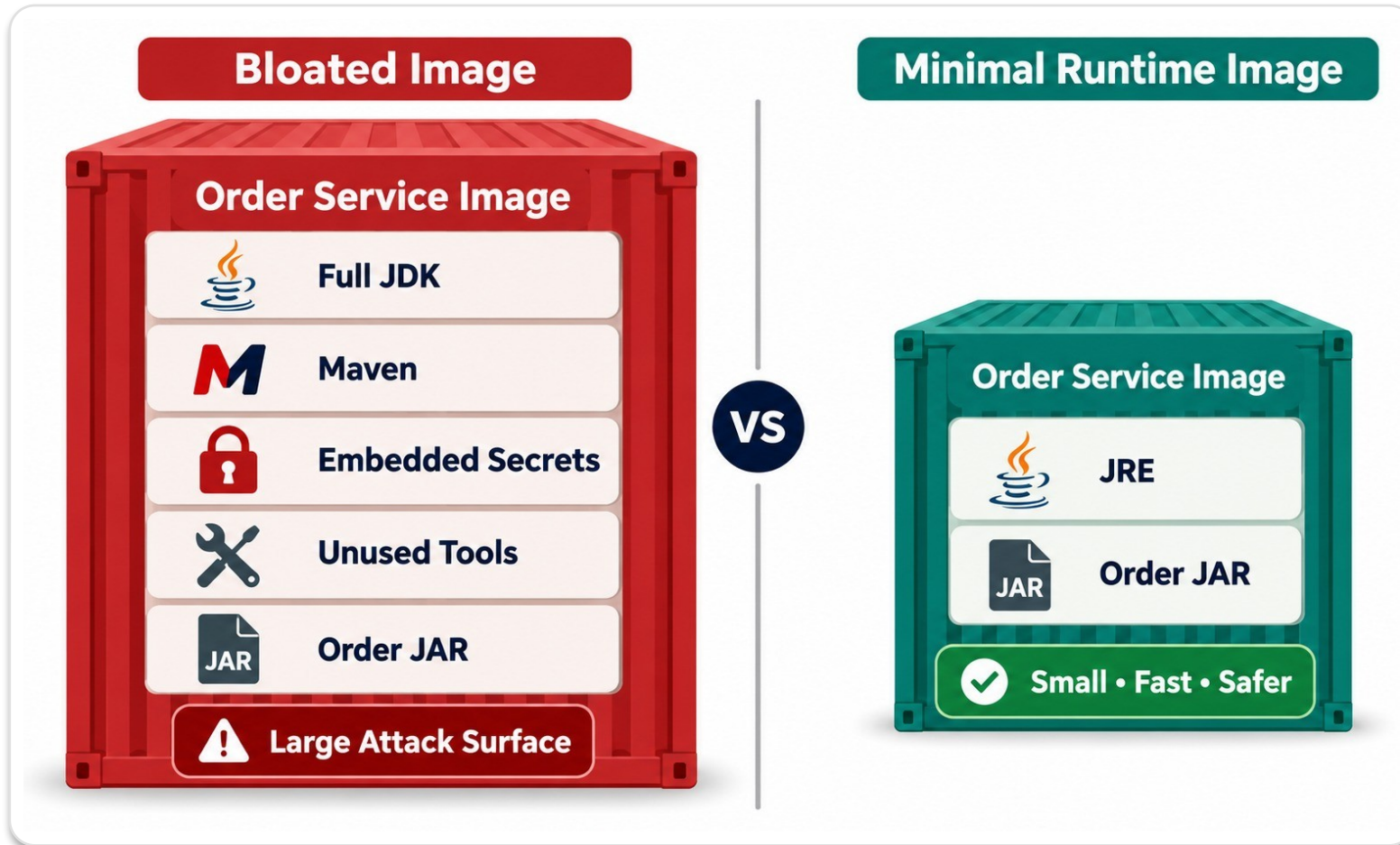
USER 1001, group 0 — fits OpenShift's random UIDs.

**TESTS** ran earlier in CI, so the image build skips them.

**KEY TAKEAWAY** Build in one stage, run in another — as a non-root user with only what the service needs.

# CONTAINER BUILD BEST PRACTICES

*Minimal base image, non-root, reproducible builds, small layers, secrets outside.*



## NEVER SHIP

- ✗ Maven, JDK and source code
- ✗ Passwords in ENV or files
- ✗ Running as root
- ✗ :latest base image

## ALWAYS

- ✓ Pinned, trusted base image
- ✓ Non-root user
- ✓ Only the runtime and JAR

**IMAGES REMEMBER** A secret in any layer stays in the image.

**KEY TAKEAWAY** Ship only what the application needs to run — and never ship secrets.

# EXERCISE 4.2 — CONTAINERIZE ACCOUNT SERVICE

*Day 4 checkpoint B: a secure, non-root image.*

OFFICIAL CHECKPOINT B

10 MIN

labs/day-04/exercises/exercise-4.2-containerize-account.md

## SCENARIO

Lab 4 ships the banking Account and Transaction services from Lab 3 as container images, then runs them with Docker Compose and on OpenShift.

## YOUR TASKS

- 1 Multi-stage build: Maven compile in the builder, JRE runtime in the final image
- 2 USER md287 (non-root); OpenShift also sets runAsNonRoot: true
- 3 Immutable tag 1.0.0 — not :latest
- 4 Build from the Lab 3 solution context — don't recopy Java trees

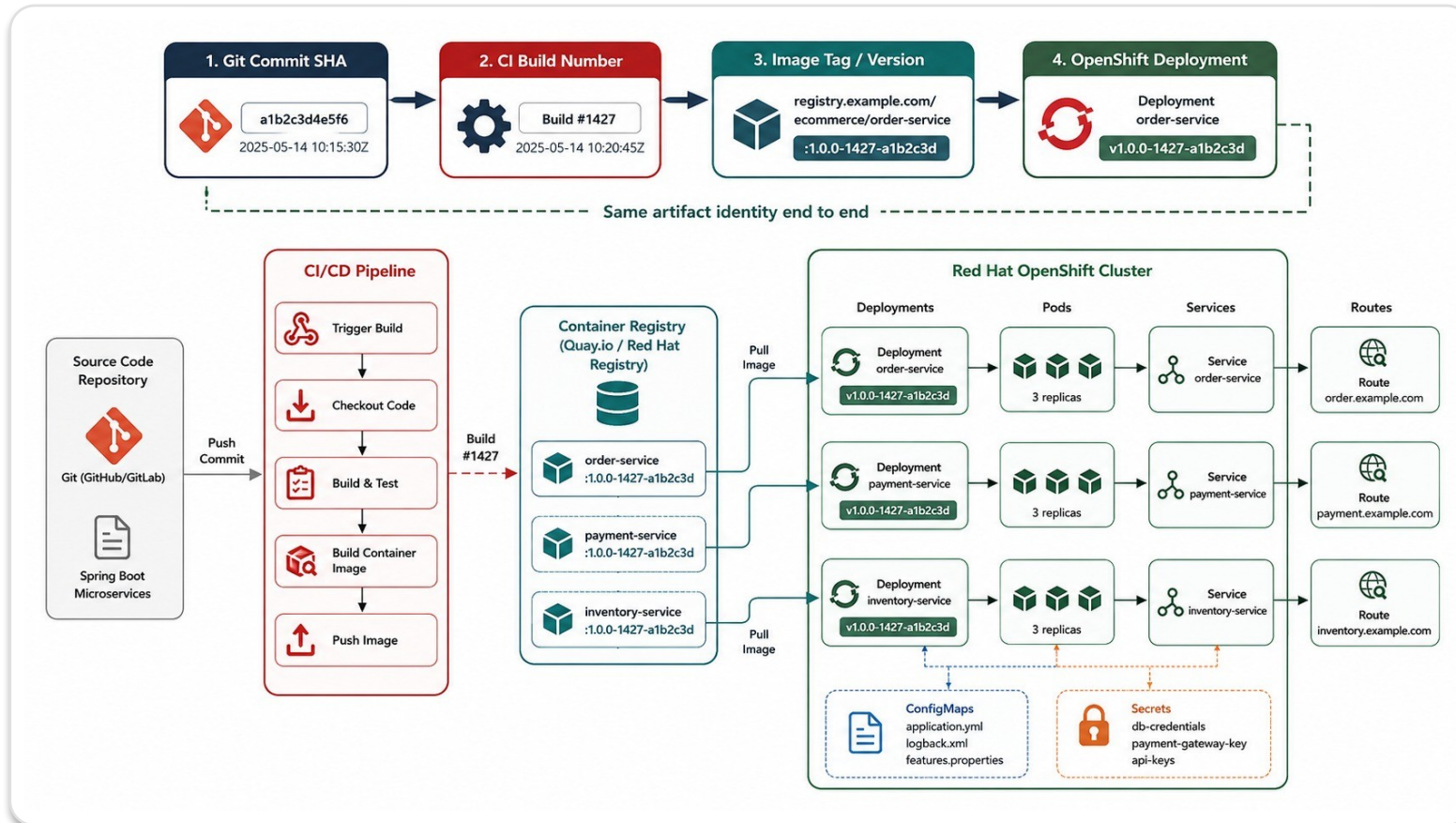
## EXPECTED OUTCOME

- ✓ Explain why the runtime user isn't root
- ✓ Say what stays in the builder stage
- ✓ Tag md287/account-service:1.0.0
- ✓ Ready before running docker build

**KEY TAKEAWAY** You can explain why the runtime user is not root before you run docker build.

# CONTAINER IMAGE TAGGING AND VERSIONING

*Semantic versions, commit SHAs and immutable references.*



## ✓ Good references

`account-service:1.0.0`  
`account-service:1.4.0-a13f92c`  
`account-service@sha256:9f2c...`

## WEAK

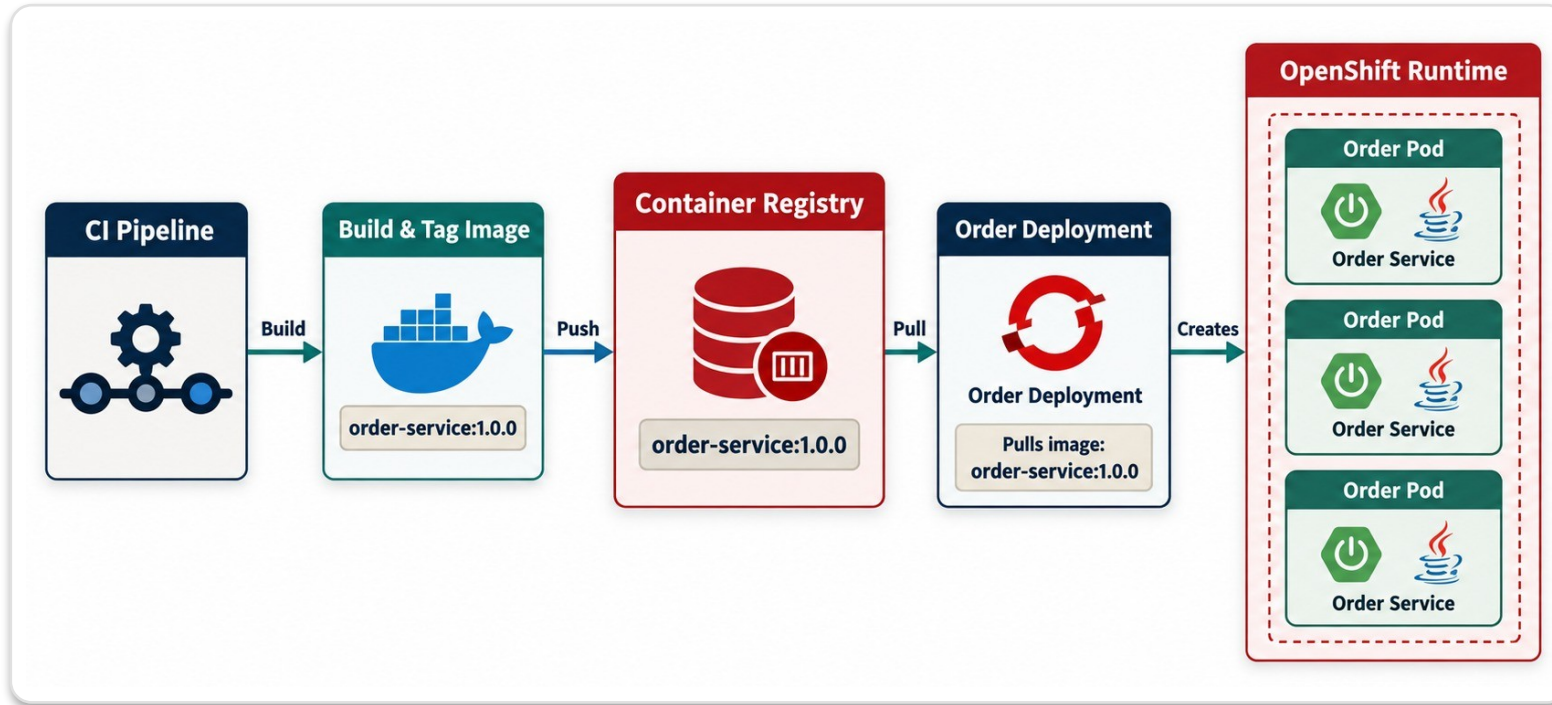
✗ `:latest` — which version is running?

**ENABLES** audits, incident analysis and exact rollback.

**KEY TAKEAWAY** Map every running deployment to an exact image and source revision.

# CONTAINER REGISTRIES

*Build → tag → push → registry → pull.*



## Push to a registry

```
IMG=quay.io/md287/account-service:1.0.0
podman login quay.io
podman tag account-service:1.0.0 $IMG
podman push $IMG
```

## Name anatomy

registry / namespace / image : tag

## EXAMPLES

Quay

OpenShift internal

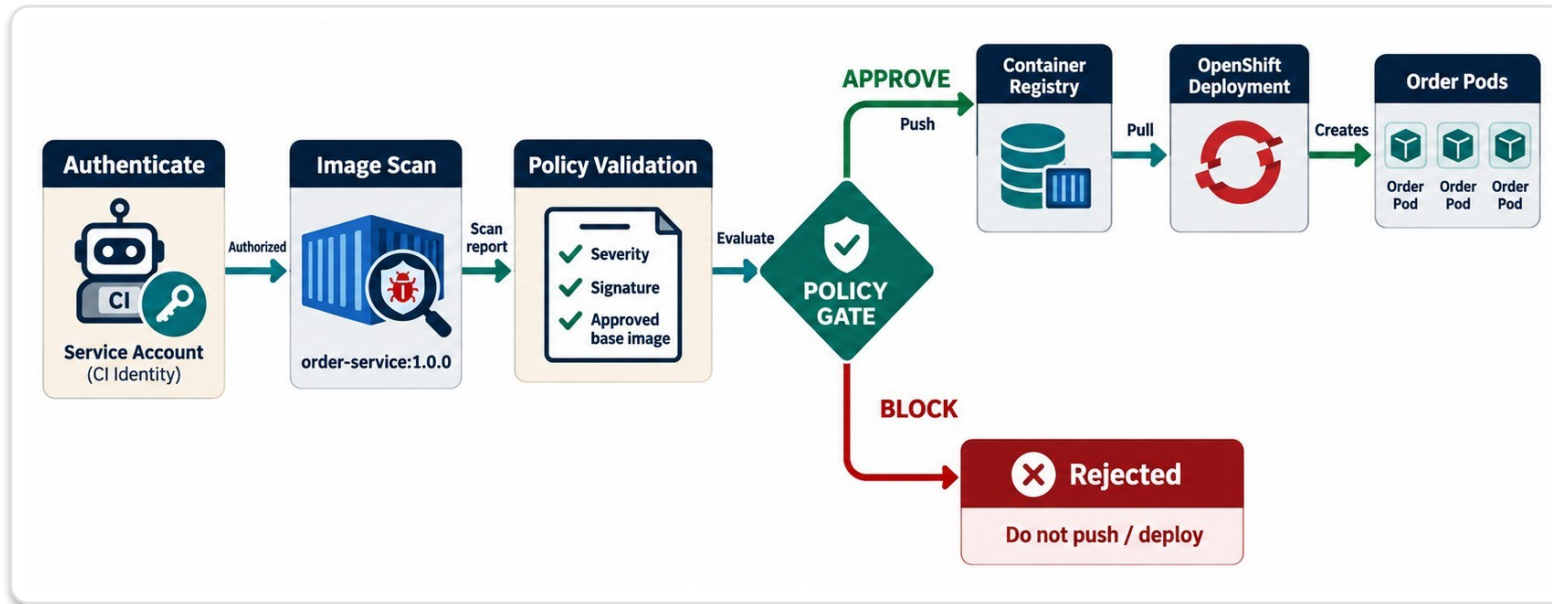
GHCR

ACR / ECR

**KEY TAKEAWAY** A registry stores and distributes the images OpenShift pulls to run pods.

# REGISTRY SECURITY AND IMAGE SCANNING

*Authentication, access control, vulnerability scanning and trusted images.*



## BEFORE PRODUCTION

- ✓ Authenticated push and pull
- ✓ Scanned for CVEs
- ✓ Signed (for example, Cosign)
- ✓ SBOM attached

### Policy gate

A CRITICAL CVE or a missing signature blocks the deploy.

**RULE** Only signed images from approved registries run in prod.

**KEY TAKEAWAY** A registry is a security boundary — authenticate, authorize, scan and gate.

# EXERCISE: REVIEW A CONTAINERFILE

*Find what makes this image slow, large and unsafe.*

## PRACTICE EXERCISE

10 MIN

### SCENARIO

A teammate wrote:

```
FROM maven:latest
COPY . /app
WORKDIR /app
ENV DB_PASSWORD=Secret123
RUN mvn package
EXPOSE 8080
CMD mvn spring-boot:run
```

### YOUR TASKS

- 1 Find at least five problems
- 2 Explain the risk of each
- 3 Sketch the improved Containerfile

### EXPECTED OUTCOME

- ✓ :latest base → pin a version
- ✓ Maven and source in runtime → multi-stage
- ✓ Password baked in → Secret at runtime
- ✓ Runs as root → USER 1001
- ✓ COPY . breaks caching → copy pom first

**KEY TAKEAWAY** Pin the base, build in stages, run as non-root, and keep secrets out of every layer.

# PART 3: OPENSIFT ARCHITECTURE

*The platform that runs, heals and exposes our containers.*



## SO FAR

- ✓ Multi-stage, non-root images
- ✓ Immutable tags trace to commits
- ✓ Registries scan and gate images

## NEXT QUESTION

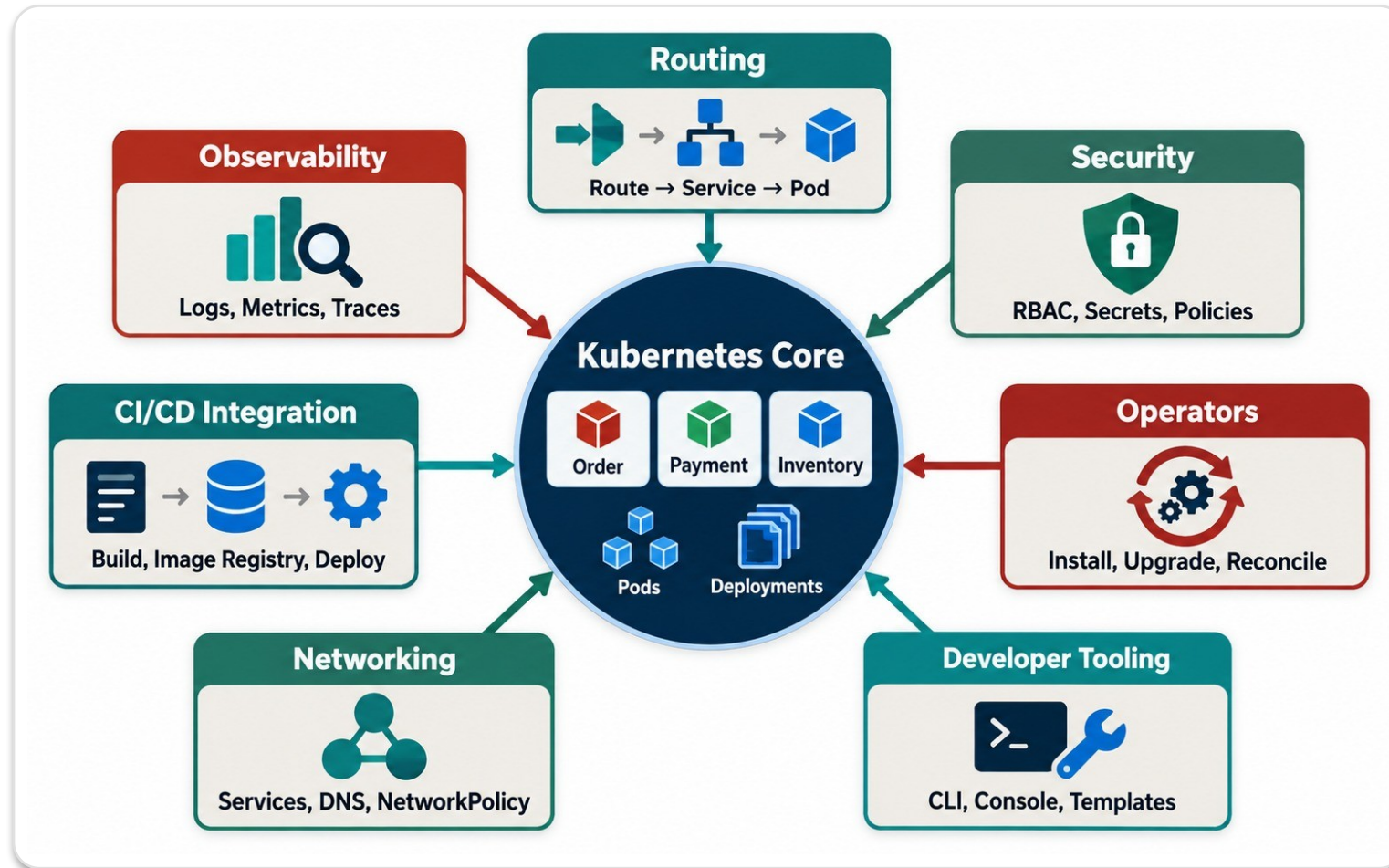
**Once the image is in the registry, what runs it — and how?**

## IN THIS PART

- 1 What Red Hat OpenShift is
- 2 Cluster architecture
- 3 Core objects and how they relate
- 4 Route → Service → Pods

# WHAT IS RED HAT OPENSHIFT?

*An enterprise Kubernetes platform with developer, security, routing and operations capabilities.*



## Kubernetes gives

Scheduling, self-healing, Services and declarative Deployments.

## OpenShift adds

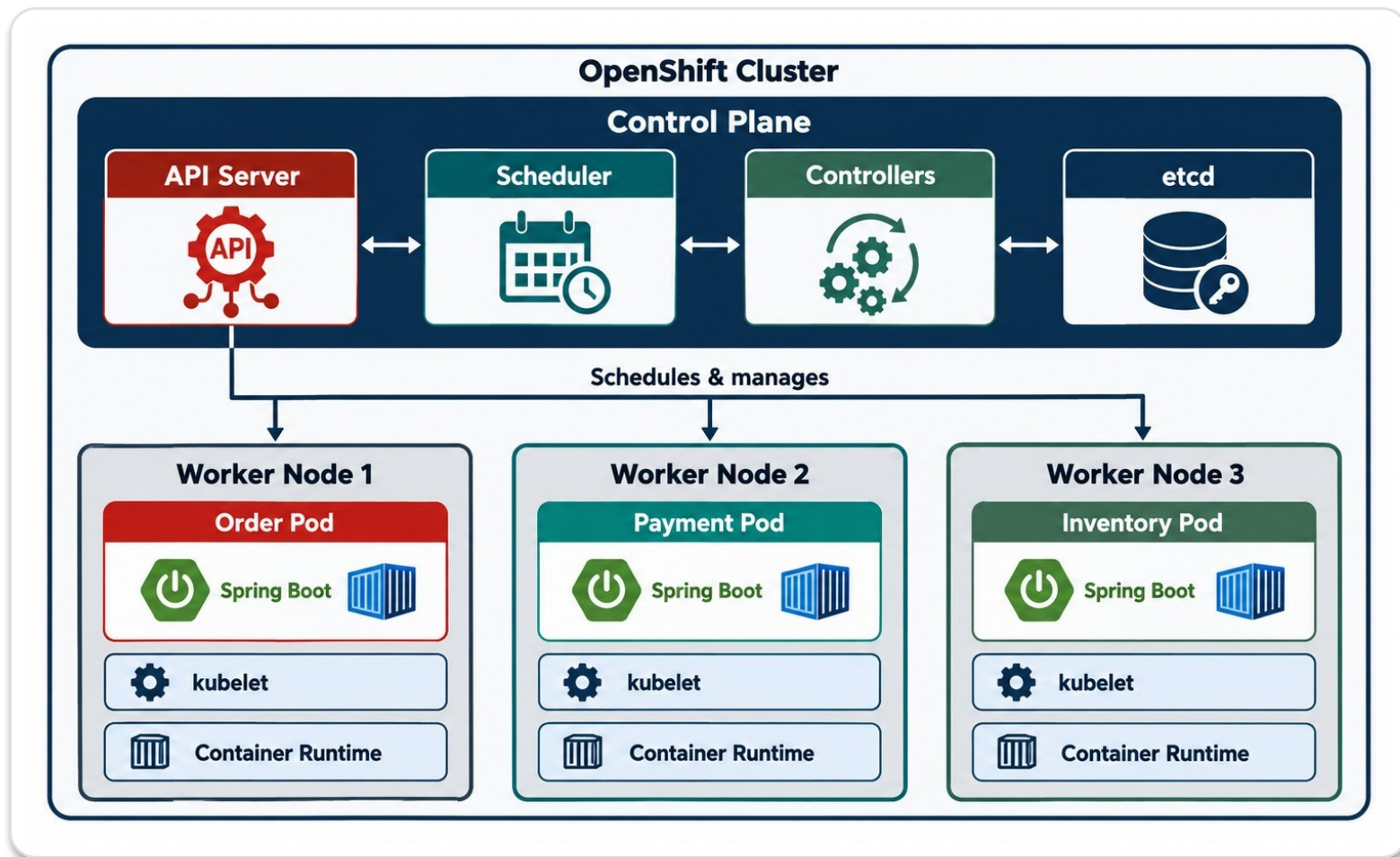
Routes, a web console, security defaults, builds, a registry and operators.

**SAME WORLD** Kubernetes YAML and kubectl skills carry over.

**KEY TAKEAWAY** OpenShift packages and extends Kubernetes into an enterprise application platform.

# OPENSIFT ARCHITECTURE OVERVIEW

*Control plane, worker nodes, API server, scheduler, etcd and operators.*



## API server

Every oc command and console action goes through it.

## Scheduler

Places pods on nodes with enough requested resources.

## etcd

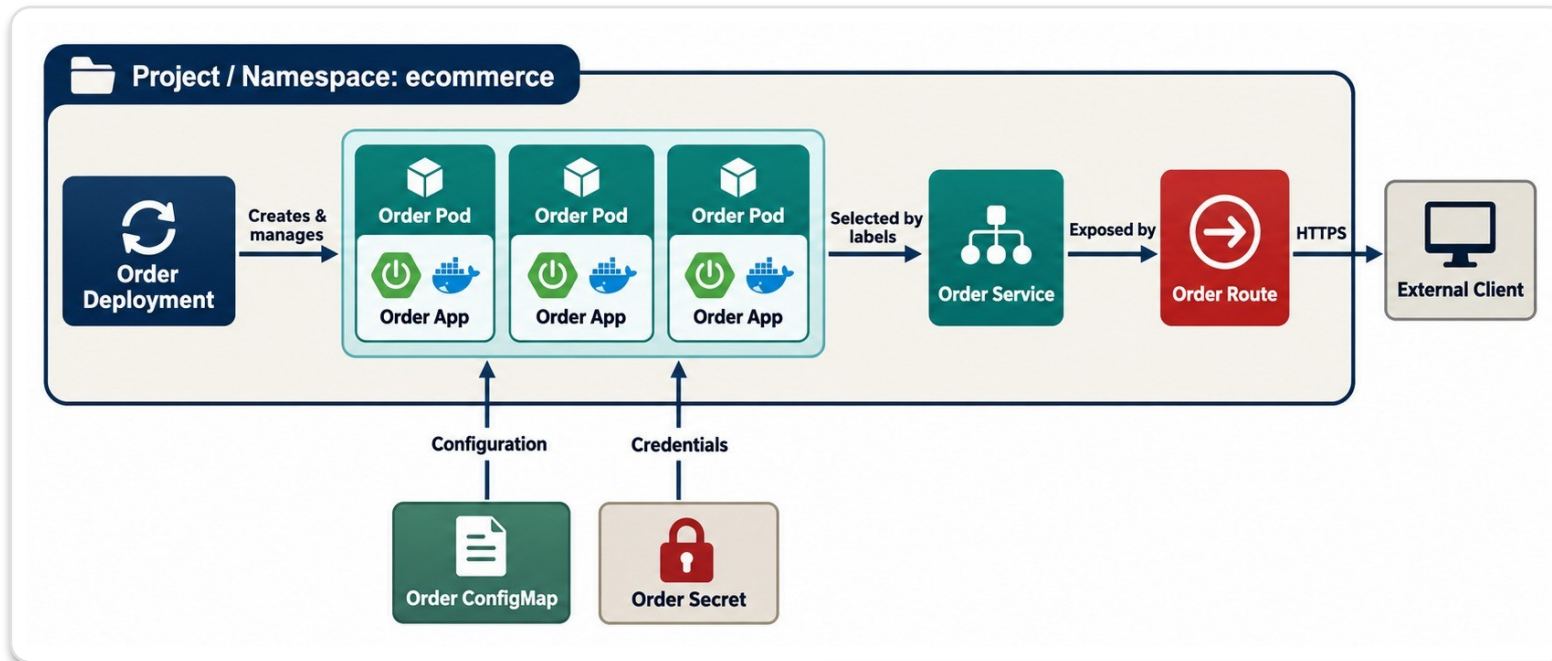
Stores the desired state of every object.

**RECONCILIATION** Controllers keep moving actual state toward desired.

**KEY TAKEAWAY** OpenShift continuously reconciles actual state toward desired state.

# CORE OPENS SHIFT OBJECTS

*Project/namespace, pod, deployment, service, route, ConfigMap and secret.*



## Project

Namespace plus access control for a team or environment.

## Deployment → Pods

Desired replicas of one image; pods are replaceable.

## Service → Route

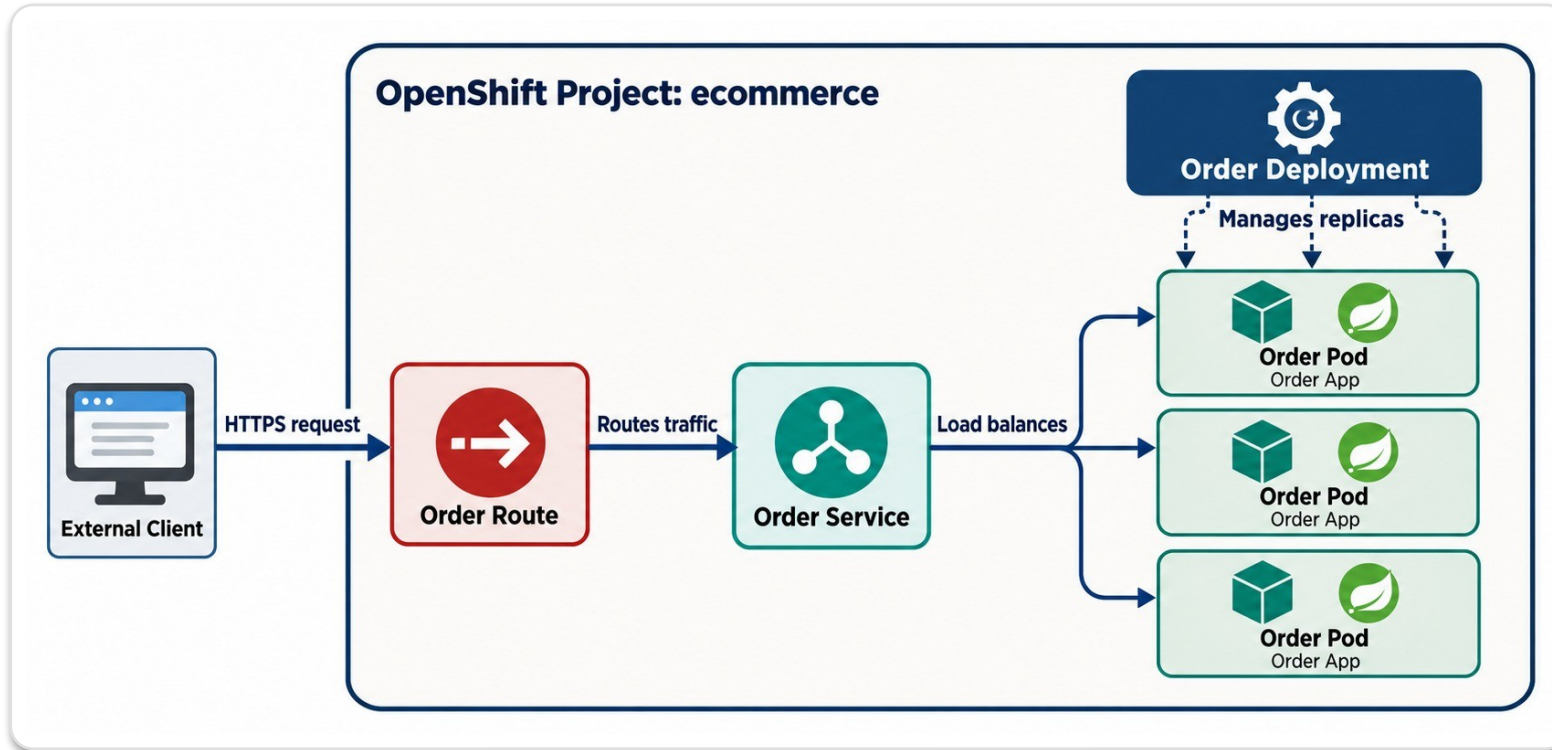
Stable internal name; optional external HTTPS URL.

**CONFIG** ConfigMaps and Secrets feed pods at runtime.

**KEY TAKEAWAY** Seven core objects describe where an app lives, how it runs, and how traffic and config reach it.

# POD, DEPLOYMENT, SERVICE, AND ROUTE

*How traffic reaches Spring Boot pods.*



## Inside the cluster — the Service name

`http://account-service:8080`

## ✓ Outside — through the Route

`https://account-service-md287.apps.example.com`

**SELECTORS** The Service finds pods by label, not by IP.

**KEY TAKEAWAY** Route → Service → Pods carries traffic, while the Deployment manages the pods.

# PART 4: RUNNING APPLICATIONS

*Deploy, configure, check, scale, update — and troubleshoot.*



## SO FAR

- ✓ OpenShift reconciles desired state
- ✓ Deployments manage replaceable pods
- ✓ Services and Routes carry traffic

## NEXT QUESTION

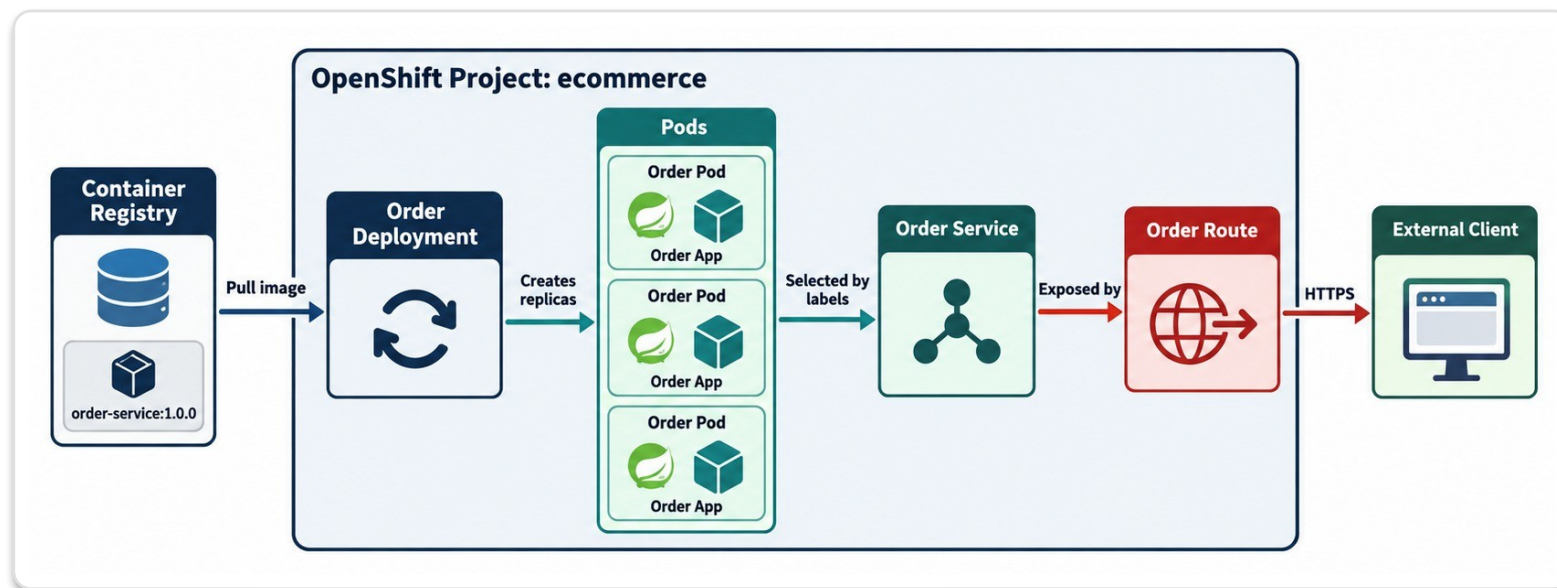
**How do we run a Spring Boot service on OpenShift so it stays healthy under change and load?**

## IN THIS PART

- 1 Deploying — with YAML
- 2 ConfigMaps and Secrets — with YAML
- 3 Probes — and Exercise 4.1
- 4 Scaling, rolling updates and resources
- 5 Troubleshooting — with an exercise
- 6 The complete picture

# DEPLOYING A MICROSERVICE TO OPENSIFT

*Image → Deployment → Pods → Service → Route.*



## Apply and watch

```
oc apply -f openshift/  
oc get pods -w  
oc get route account-service
```

## What OpenShift does

Pulls the image, schedules pods, runs probes, then routes traffic.

**DECLARATIVE** Change the YAML in Git;  
reapply — don't edit live.

**KEY TAKEAWAY** Declare the image and replicas; OpenShift schedules, starts, checks and exposes the pods.

# DEPLOYMENT, SERVICE AND ROUTE

*Three manifests take Account Service from image to HTTPS URL.*

## 10-account.yaml (Deployment)

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: account-service }
spec:
  replicas: 2
  selector: { matchLabels: { app: account-service } }
  template:
    metadata: { labels: { app: account-service } }
    spec:
      securityContext: { runAsNonRoot: true }
      containers:
        - name: account-service
          image: <registry>/md287/account-service:1.0.0
          ports: [ { containerPort: 8080 } ]
          envFrom:
            - configMapRef: { name: account-config }
            - secretRef: { name: account-secret }
```

### Labels

app: account-service ties the Service to the pods.

### envFrom

Config and secrets arrive as environment variables.

## ✓ Service and Route

```
apiVersion: v1
kind: Service
metadata: { name: account-service }
spec:
  selector: { app: account-service }
  ports: [ { port: 8080, targetPort: 8080 } ]
---
apiVersion: route.openshift.io/v1
kind: Route
metadata: { name: account-service }
spec:
  to: { kind: Service, name: account-service }
  port: { targetPort: 8080 }
  tls: { termination: edge }
```

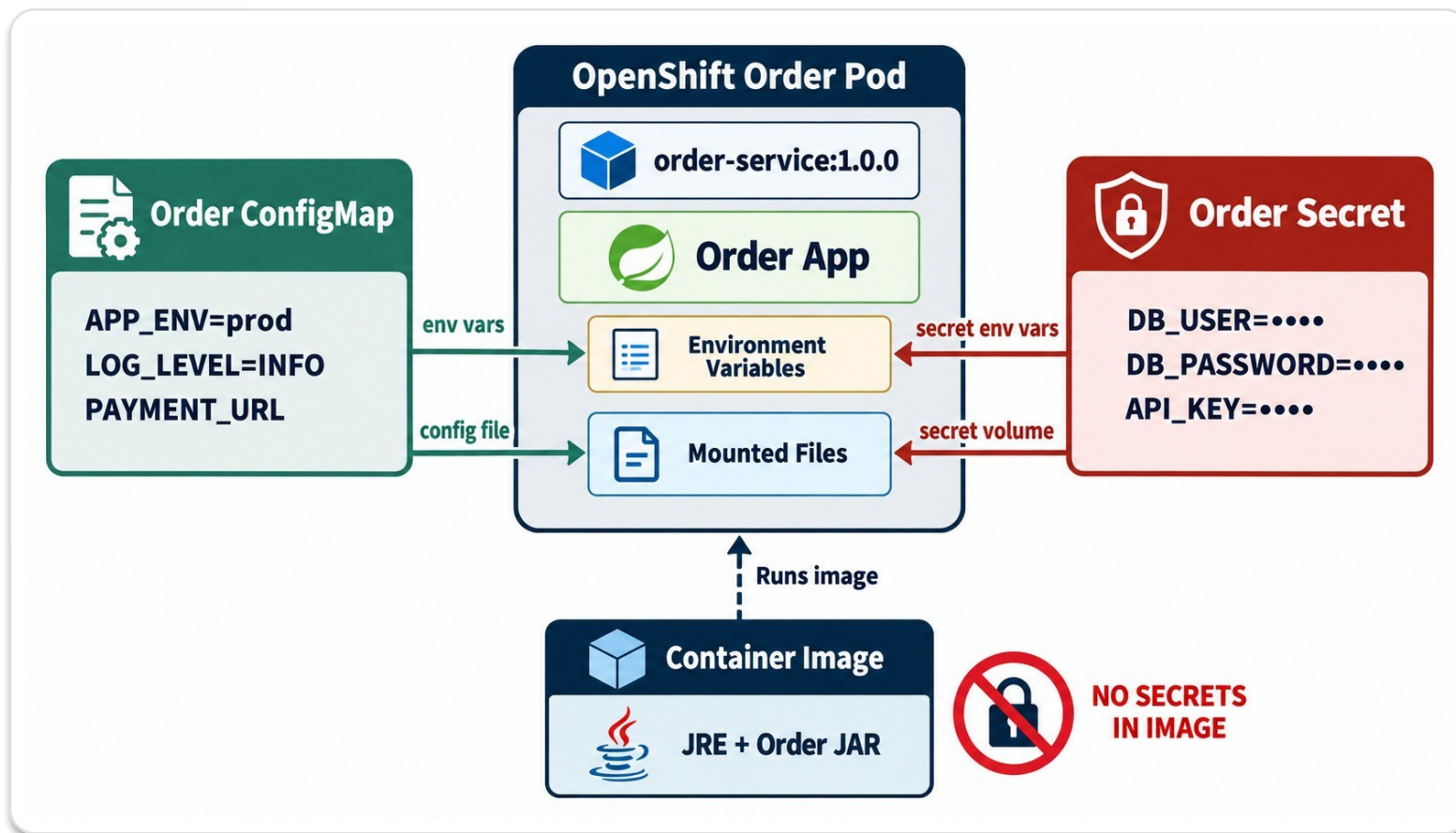
### Route

Edge TLS — HTTPS outside, plain HTTP to the pod.

**KEY TAKEAWAY** A Deployment runs the pods, a Service names them, and a Route exposes only what must be public.

# CONFIGURATION AND SECRETS IN OPENSHIFT

*ConfigMaps, secrets, environment variables and mounted files.*



## ConfigMap

JDBC URL, Kafka bootstrap servers, feature flags.

## Secret

DB password, JWT signing secret, API keys — tightly restricted.

## Spring binding

`SPRING_DATASOURCE_URL` maps to `spring.datasource.url` automatically.

**NOTE** Secrets are base64 — restrict who can read them.

**KEY TAKEAWAY** Same image + environment-specific ConfigMaps and secrets — for DEV, TEST and PROD.

# CONFIGMAP AND SECRET FOR ACCOUNT SERVICE

*Non-sensitive settings in a ConfigMap; credentials in a Secret — never in the image.*

## ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata: { name: account-config }
data:
  SPRING_PROFILES_ACTIVE: openshift
  SPRING_DATASOURCE_URL: jdbc:postgresql://account-db:5432/account_db
  SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:19092
```

## ✓ Secret

```
apiVersion: v1
kind: Secret
metadata: { name: account-secret }
type: Opaque
stringData:
  SPRING_DATASOURCE_USERNAME: account_app
  SPRING_DATASOURCE_PASSWORD: <from the vault>
  MD287_JWT_SECRET: <from the vault>

# apply from a secure source – not from Git
```

### Relaxed binding

Environment names map to Spring properties.

### Per environment

Each project gets its own values; the image is unchanged.

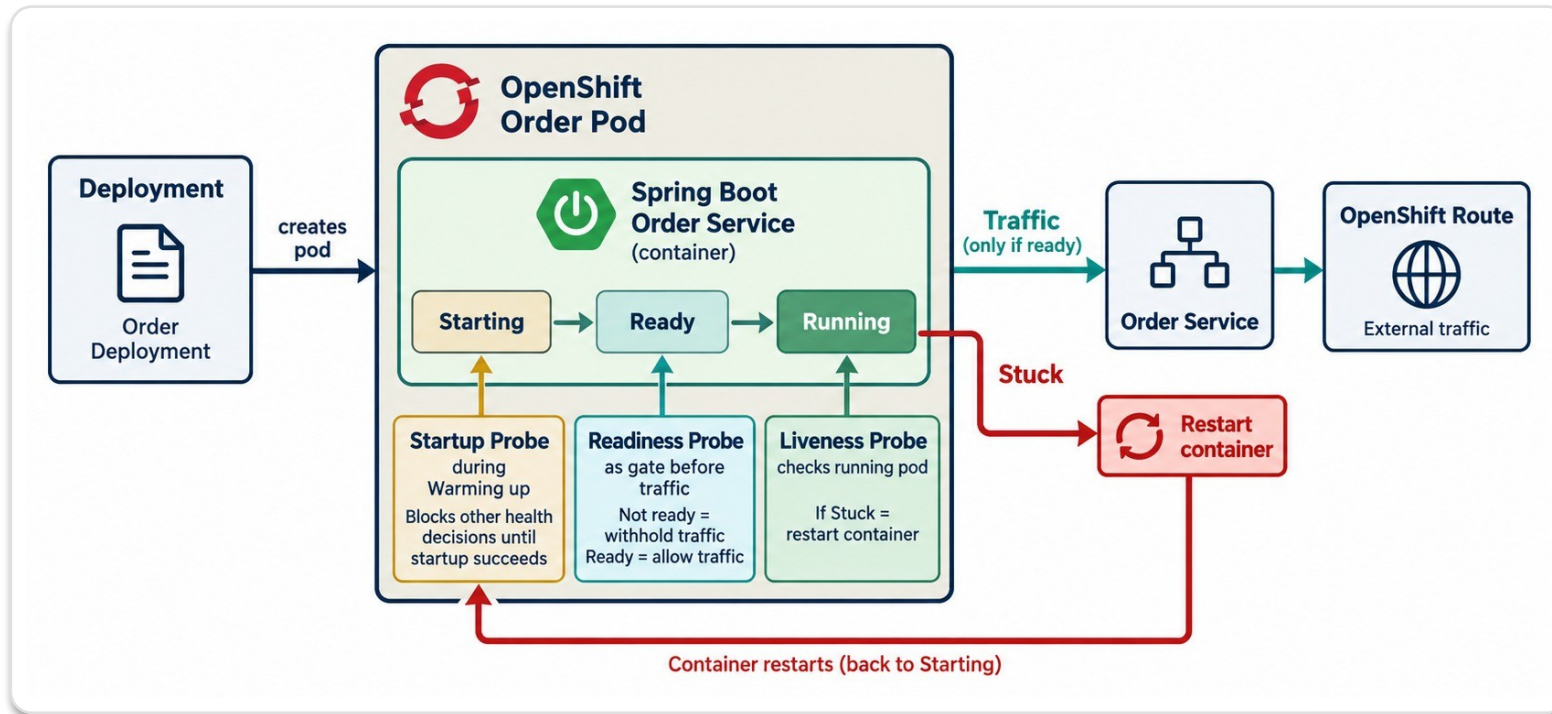
### Secrets in Git?

No — use a vault, sealed or external secrets.

**KEY TAKEAWAY** Change behavior per environment with ConfigMaps and Secrets — the image stays exactly the same.

# HEALTH CHECKS IN OPENSSHIFT

*Startup, readiness and liveness probes.*



## Startup

Gives Spring Boot time to boot before other probes start.

## Readiness

Fails → the pod is removed from the Service, not restarted.

## Liveness

Fails repeatedly → OpenShift restarts the container.

**WARNING** Never make liveness depend on the database.

**KEY TAKEAWAY** Probes tell OpenShift when a pod has started, can take traffic, or needs a restart.

# PROBES BACKED BY SPRING BOOT ACTUATOR

*Actuator's health groups become OpenShift's startup, readiness and liveness checks.*

## application.yml

```
management:
  endpoint:
    health:
      probes:
        enabled: true
  endpoints:
    web:
      exposure:
        include: health, info, metrics

# /actuator/health/liveness
# /actuator/health/readiness
```

## ✓ Container probes

```
startupProbe:
  httpGet: { path: /actuator/health/liveness, port: 8080 }
  periodSeconds: 5
  failureThreshold: 30           # up to 150 s to boot
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  periodSeconds: 10
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 8080 }
  periodSeconds: 15
  failureThreshold: 3
```

### Startup

Holds off liveness until the app has booted.

### Readiness

Controls whether the Service sends traffic.

### Security

Health stays public; metrics still need a JWT.

**KEY TAKEAWAY** Let Actuator report health and OpenShift act on it — start, route, or restart.

# EXERCISE 4.1 — ADD HEALTH PROBES AND CORRELATION LOGGING

*Day 4 checkpoint A: operability basics.*

OFFICIAL CHECKPOINT A

10 MIN

labs/day-04/exercises/exercise-4.1-probes-correlation.md

## SCENARIO

Lab 4 runs Account and Transaction services with Docker Compose, then deploys them to OpenShift with the same probes.

## YOUR TASKS

- 1 Liveness vs. readiness vs. startup — which Actuator paths, and why startup exists
- 2 X-Correlation-Id on the request must appear as correlationId= in logs
- 3 Health stays public; /actuator/metrics still needs a JWT
- 4 Tracing (OpenTelemetry) is a demonstration — no Jaeger to stand up

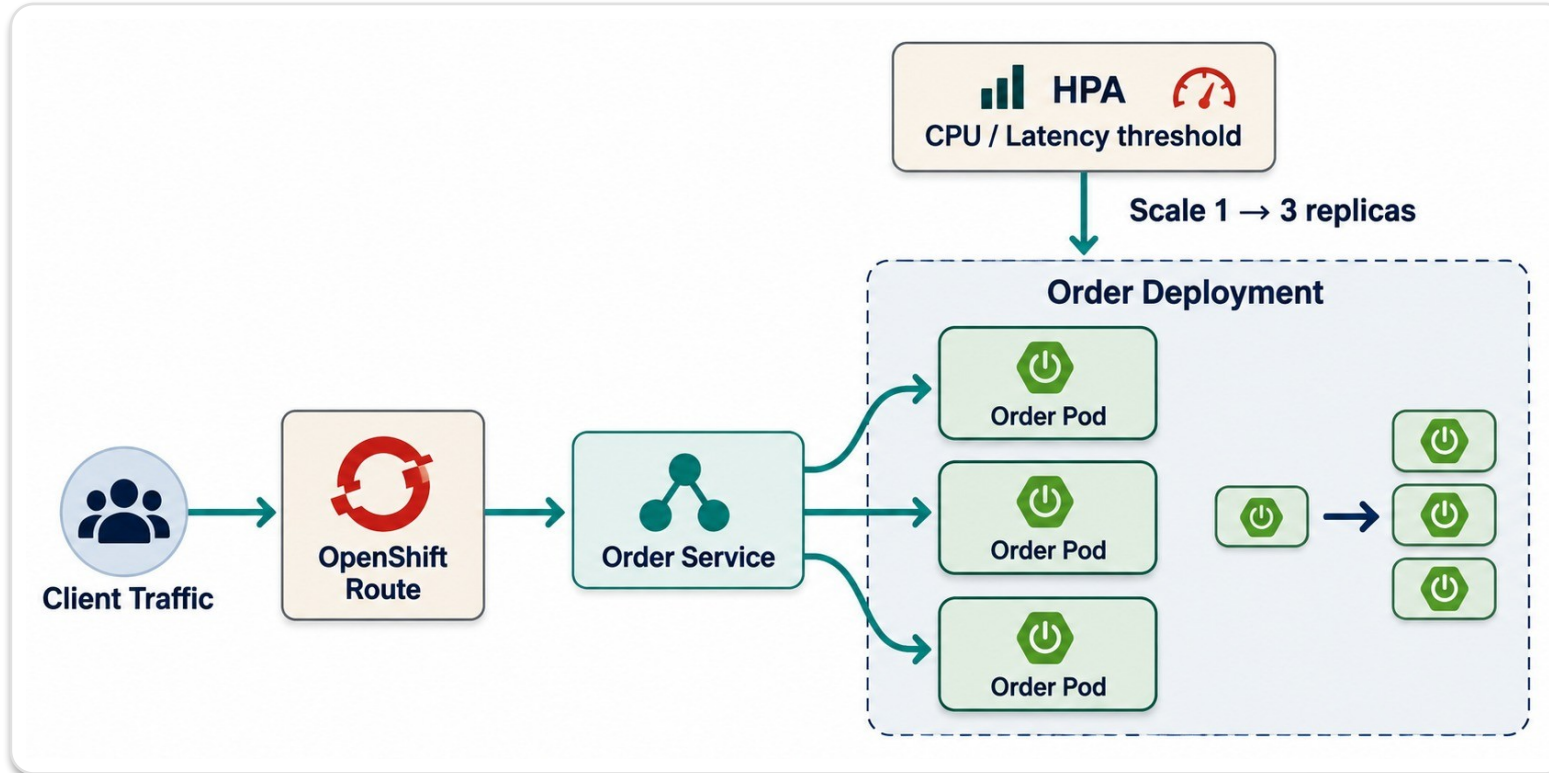
## EXPECTED OUTCOME

- ✓ Tell liveness from readiness
- ✓ Explain why startup probes exist
- ✓ Find a correlationId in a log line
- ✓ Know which endpoints need a token

**KEY TAKEAWAY** You can tell liveness from readiness and find a correlation ID in a log line.

# SCALING APPLICATIONS IN OPENSIFT

*Replica scaling and horizontal scaling concepts.*



## Manual and automatic

```
oc scale deploy/account-service \
  --replicas=4
```

```
oc autoscale deploy/account-service \
  --min=2 --max=8 --cpu-percent=70
```

## CHECK BEFORE SCALING OUT

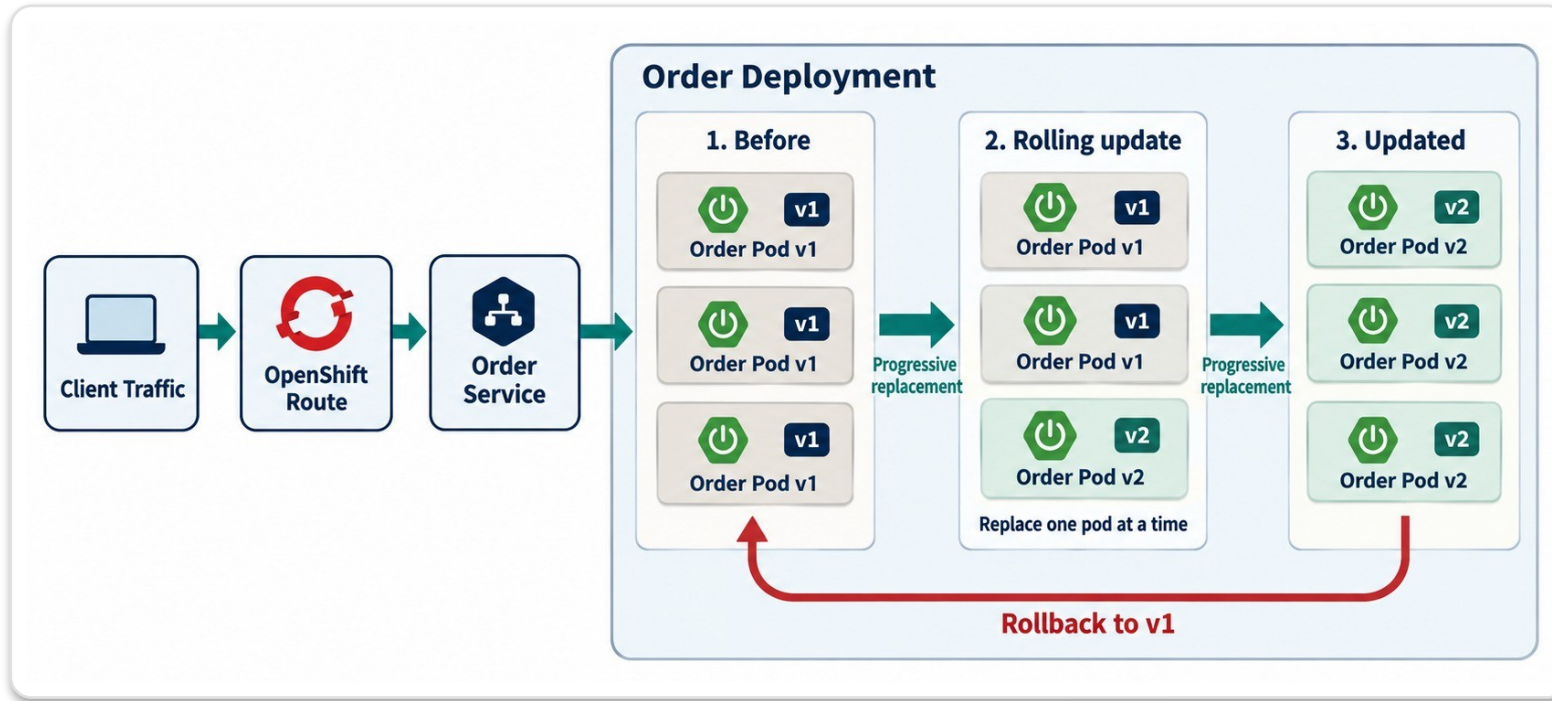
- ! Stateless pods
- ! Database connection limits
- ! Downstream capacity

**HPA** needs CPU requests set to calculate utilization.

**KEY TAKEAWAY** Scale out by adding replicas — but check stateless design and downstream capacity.

# ROLLING UPDATES AND ROLLBACKS

*Progressive pod replacement, readiness gates and reverting failed releases.*



## Roll forward, then back

```
oc set image deploy/account-service \
  account-service=$IMG:1.0.1
oc rollout status deploy/account-service
oc rollout undo deploy/account-service
```

## Readiness gate

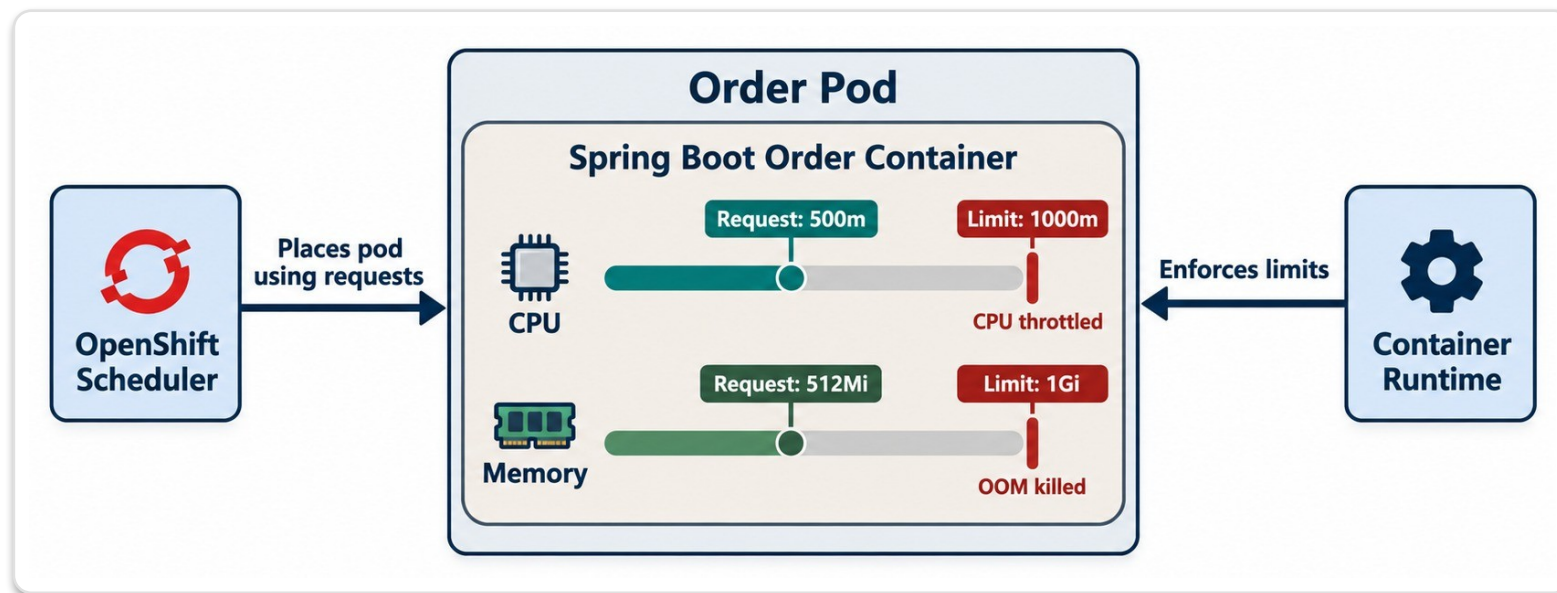
Old pods keep serving until new ones are ready.

**LIMIT** Rollback restores the image — not the database schema.

**KEY TAKEAWAY** Rolling updates replace pods gradually; readiness keeps traffic safe during the rollout.

# MANAGING RESOURCES AND APPLICATION LIMITS

*CPU and memory requests and limits, scheduling and stability.*



## A starting point

```
resources:  
  requests: { cpu: 250m, memory: 512Mi }  
  limits:   { cpu: "1",   memory: 1Gi }
```

## Over the memory limit

The container is OOM-killed and restarted.

## Over the CPU limit

The container is throttled — slower, not killed.

**MEASURE FIRST** Set values from real usage, then adjust.

**KEY TAKEAWAY** Requests guide scheduling, limits cap usage — both should come from measurement.

# OBSERVING AND MANAGING APPLICATIONS

*Logs, events, status, metrics and troubleshooting.*



## Walk the path

```
oc get route,svc,endpoints
oc get pods
oc describe pod <pod>      # events
oc logs <pod> --previous
```

## COMMON STATES

ImagePullBackOff

CrashLoopBackOff

OOMKilled

0/2 ready

**RULE** Read events and logs before restarting anything.

**KEY TAKEAWAY** Troubleshoot systematically along the traffic path, not by restarting first.

# EXERCISE: WHY CAN'T USERS REACH THE API?

*Troubleshoot a deployed service along the traffic path.*

## PRACTICE EXERCISE

10 MIN

### SCENARIO

Account Service is deployed. `oc get pods` shows the pod Running, but users get an error from the Route URL.

```
$ oc get pods
account-service-7c9d 0/1 Running 0 4m

$ oc get endpoints account-service
account-service <none>
```

### YOUR TASKS

- 1 List what to check, in order
- 2 Explain what 0/1 and <none> tell you
- 3 Name two likely causes and their fixes

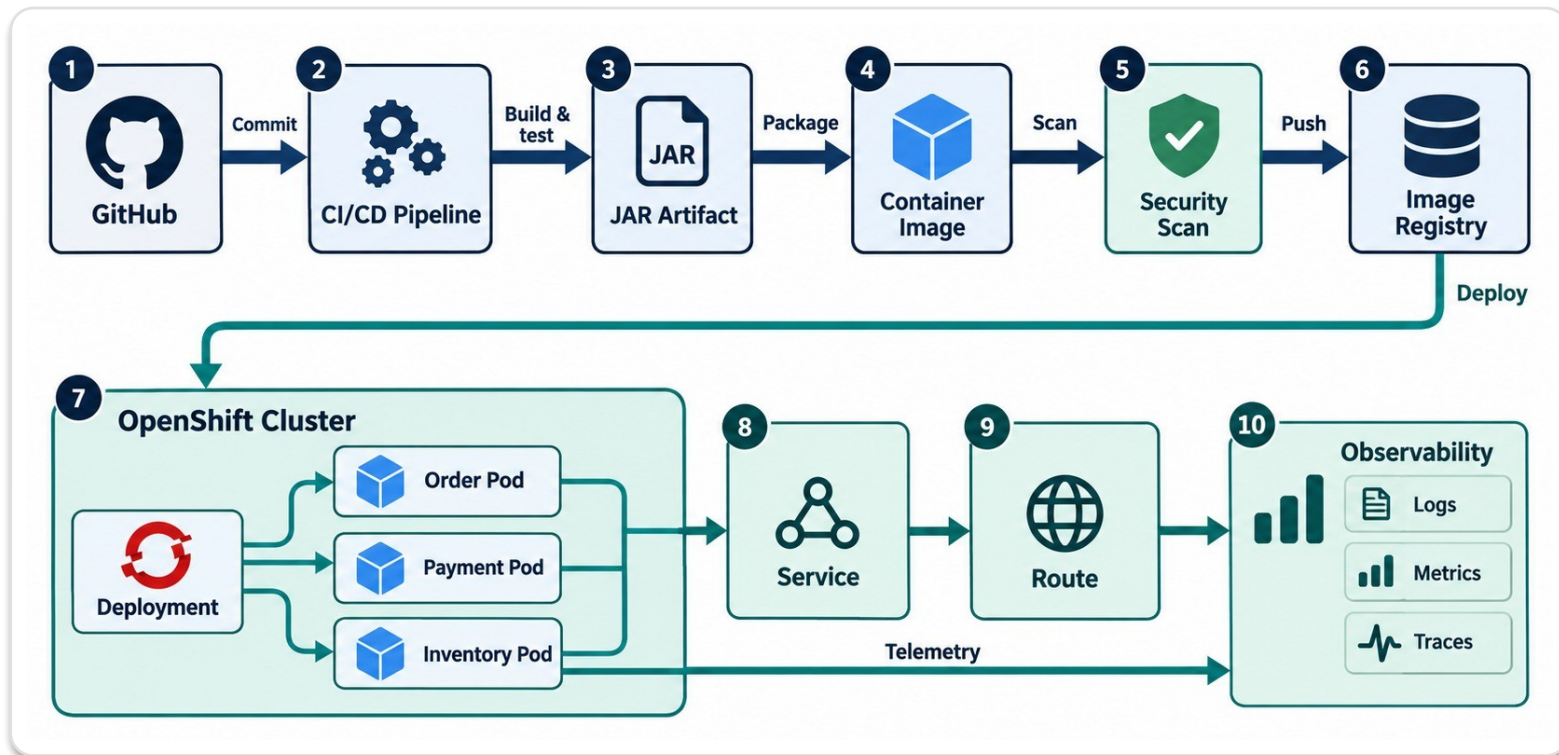
### EXPECTED OUTCOME

- ✓ Route → Service → endpoints → pod readiness
- ✓ 0/1: running but not ready; no endpoints
- ✓ Readiness failing — a DB or config issue
- ✓ Or the Service selector doesn't match the labels
- ✓ Check describe events and logs

**KEY TAKEAWAY** Running isn't ready: follow Route, Service, endpoints, readiness, events and logs — in that order.

# OPENSIFT MICROSERVICES DEPLOYMENT WORKFLOW

*Git → CI/CD → registry → OpenShift → service and route → monitoring.*



## Module 7 half

Git, CI, tests, scans, image, SBOM, signature and registry.

## Module 8 half

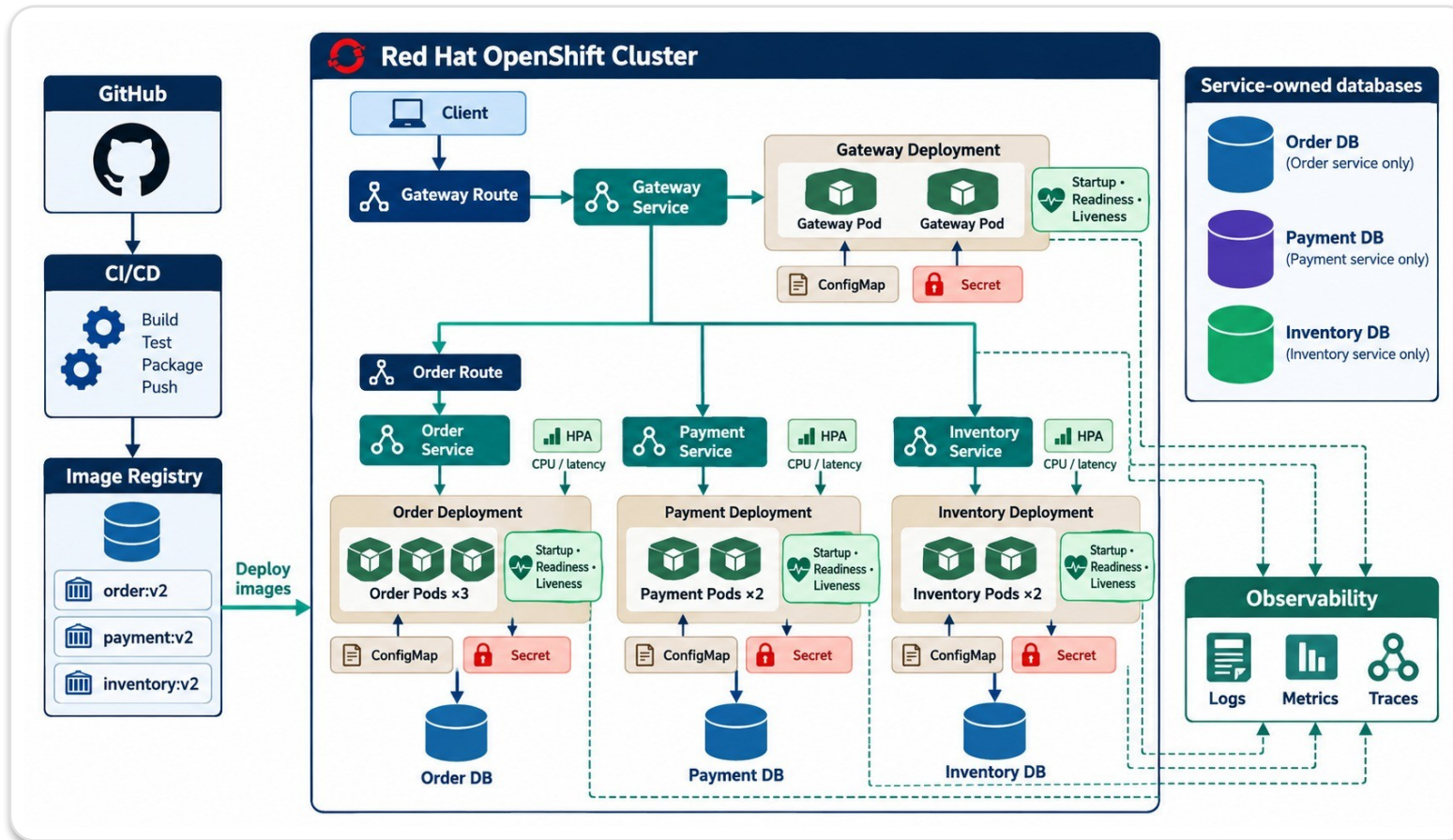
Deployment, pods, probes, Service, Route, scaling and rollout.

**LAB 4** walks this workflow on the classroom cluster.

**KEY TAKEAWAY** Git and CI/CD produce the release; OpenShift runs it; observability feeds back.

# END-TO-END SPRING BOOT MICROSERVICES ON OPENSHIFT

*Containers, registry, OpenShift, scaling, config, health and routing together.*



## EVERY PART OF THE MODULE

- ✓ Non-root images from the registry
- ✓ Deployments with replicas and resources
- ✓ ConfigMaps and Secrets per environment
- ✓ Startup, readiness and liveness probes
- ✓ Services inside, Routes only at the edge

**REMEMBER** Same image everywhere; the platform does the rest.

**KEY TAKEAWAY** Containers package the service; OpenShift runs, scales, updates and manages it reliably.

# CONTAINER AND OPENSIFT MISCONCEPTIONS

*Beliefs and habits that make platform deployments fragile or unsafe.*

## MISCONCEPTION

## REALITY

## ANTI-PATTERNS TO AVOID

**Containers are lightweight VMs**

They're isolated processes on a shared kernel

**latest is fine for deployments**

Immutable tags make releases traceable

**Running means healthy**

Only readiness means it can take traffic

**Secrets are encrypted by default**

Base64 isn't encryption — restrict access

**Rollback also restores the data**

It restores the image, not the schema

- ✗ Root user in the image
- ✗ Secrets baked into layers
- ✗ No resource requests or limits
- ✗ Liveness checks that call the database
- ✗ A Route for every internal service
- ✗ Editing live objects instead of Git

**REMEMBER** Same image, external config, honest probes.

**KEY TAKEAWAY** Reliable platforms come from immutable images, runtime config and probes that tell the truth.

# KNOWLEDGE CHECK

*Answer in your own words — then compare with a partner.*

1 Container image vs. container?

2 Why are containers useful for microservices?

3 What is a Dockerfile or Containerfile for?

4 Why should image tags be traceable?

5 What is a container registry?

6 What is an OpenShift project?

7 What does a Deployment manage?

8 Service vs. Route?

9 When use a Secret instead of a ConfigMap?

10 Why are readiness probes important?

# MODULE 8 SUMMARY AND WHAT'S NEXT

*From a Spring Boot JAR to a healthy, scalable service on OpenShift.*

## YOU CAN NOW

- ✓ Explain containers, images and how they differ from VMs
- ✓ Write a multi-stage, non-root Containerfile
- ✓ Tag, push and gate images in a registry
- ✓ Deploy with Deployments, Services, Routes, ConfigMaps and Secrets
- ✓ Use probes, scaling, rollouts and a troubleshooting path

## NEXT: MODULE 9 — GITHUB COPILOT FOR SPRING BOOT

- 1 How can AI speed up Spring Boot development safely?
- 2 What makes a good prompt for backend code?
- 3 Who stays responsible for AI-generated code?

**BRING ALONG** Your Lab 4 services and manifests — they're the context for Copilot.

**KEY TAKEAWAY** Module 9 uses AI to build these services faster — without lowering the bar.